Linear-Quadratic Optimal Reinforcement Learning Controller Design in Discrete Time for a Triple Inverted Pendulum

Author: Cambukingham

October 14, 2026

Abstract

This report designs a discrete-time linear-quadratic (LQ) optimal controller for a six-state triple inverted pendulum, a model also used for humanoid balance. The optimal gain is obtained by Hewer's algorithm, a reinforcement learning (policy iteration) scheme in which a Lyapunov equation evaluates a policy and a gain update improves it. The iterates are compared with the solution of the discrete algebraic Riccati equation (DARE). Starting from a pole-placement gain with cost 6.57×10^6, the cost reaches the optimal value $J^* \approx 31090.76$ within six iterations.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Reinforcement Learning from an Engineering View . . . 3
- 1.3 Objectives and Organization . . . 3

2 System Model **3**
- 2.1 Background: Nonlinear Dynamics . . . 3
- 2.2 Linearization . . . 3
- 2.3 Variables . . . 4

3 The Linear-Quadratic Optimal Control Problem **4**
- 3.1 Background . . . 4
- 3.2 Performance Criterion . . . 4
- 3.3 Control Law . . . 4
- 3.4 Existence Conditions . . . 5
- 3.5 Dynamic Programming and the DARE . . . 5

4 The Lyapunov Equation and Policy Evaluation **6**
- 4.1 Background: Lyapunov Stability . . . 6
- 4.2 Cost of a Fixed Gain . . . 6

5 Hewer's Algorithm **7**
- 5.1 Background: Policy Iteration . . . 7
- 5.2 Derivation . . . 7
- 5.3 Properties . . . 7

6 Problem Setup and Implementation **8**
6.1 Model and Checks 8
6.2 Initial Stabilizing Gain 8
6.3 Initial Condition and Metrics 8

7 Results **8**
7.1 Convergence Table 8
7.2 Controls and Trajectories 10

8 Discussion **10**
8.1 Other Controllers 13

9 Conclusions **13**

A MATLAB Code **13**

B Model Data **16**

List of Figures

1 State-feedback control loop. The closed-loop matrix is $A_d - B_d F$. 5
2 Approximate feedback controls u_1 and u_2 for $k = 1, \ldots, 6$ compared with the optimal control from `dlqr` (dashed). 9
3 State x_4 for $k = 1, \ldots, 6$ and the optimal gain (dashed). 10
4 State x_5 for $k = 1, \ldots, 6$ and the optimal gain (dashed). 11
5 State x_6 for $k = 1, \ldots, 6$ and the optimal gain (dashed). 12

List of Tables

1 Variables of the state space model. 4
2 Convergence of the Hewer iterations to the optimal performance value. 9
3 Notation used in this report. 16

1 Introduction

1.1 Motivation

Balancing is one of the central problems in legged robotics. A standing humanoid is statically unstable: left alone, small deviations from the upright posture grow with time. The triple (three-link) inverted pendulum captures this behavior in its simplest useful form. Its links represent body segments (for example shank, thigh, and trunk) joined at ankle, knee, and hip joints, so the same model is used to study posture control of humanoid robots [1, 2].

1.2 Reinforcement Learning from an Engineering View

Reinforcement learning (RL) improves a decision rule (a *policy*) by evaluating how well it performs and then adjusting it. For linear systems with a quadratic cost, this idea can be written in closed form: the policy is a state-feedback gain, evaluating it means solving a Lyapunov equation, and improving it means one matrix update. Hewer's algorithm [3, 4] repeats these two steps and converges to the LQ optimal controller.

1.3 Objectives and Organization

The objectives are to (i) discretize the linearized pendulum, (ii) verify the conditions for the LQ solution to exist, (iii) run Hewer's algorithm and show that it converges to the DARE solution, and (iv) examine the controls and state trajectories of the intermediate gains. Section 2 presents the model, Section 3 the LQ problem and the DARE, Section 4 the Lyapunov equation, Section 5 the Hewer algorithm and its link to Newton's method, Section 6 the problem setup, and Sections 7 and 8 the results and discussion. Derivations and code are in the appendices.

2 System Model

2.1 Background: Nonlinear Dynamics

A chain of n rigid links on a pivot is described by the Euler–Lagrange equations. For the triple pendulum with joint angles $\boldsymbol{\theta} = [\theta_1\ \theta_2\ \theta_3]^\top$ and joint torques $\boldsymbol{\tau}$,

$$\mathbf{M}(\boldsymbol{\theta})\,\ddot{\boldsymbol{\theta}} + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\,\dot{\boldsymbol{\theta}} + \mathbf{G}(\boldsymbol{\theta}) = \boldsymbol{\tau}, \tag{2.1}$$

where $\mathbf{M}$ is the inertia matrix, $\mathbf{C}$ collects Coriolis and centripetal terms, and $\mathbf{G}$ is the gravity vector.

2.2 Linearization

LQ theory needs linear dynamics. The upright position is an equilibrium and the controller's goal is to keep the state near it, so Eq. (2.1) is linearized there using $\sin\theta \approx \theta$ and $\cos\theta \approx 1$, and dropping products of small quantities. The result is a linear model with a six-dimensional state and two control inputs,

$$x \in \mathbb{R}^6, \qquad u \in \mathbb{R}^2, \tag{2.2}$$

$$\dot{x}(t) = A\,x(t) + B\,u(t). \tag{2.3}$$

The numerical A and B are taken from [1] and listed in Appendix B. The six states and the two inputs are as defined in [1]: θ_1: Angular displacement of the first arm, θ_2: Angular displacement of

the second arm, θ_3: Angular displacement of the third arm, $\dot{\theta}_1$: Angular velocity of the first arm, $\dot{\theta}_2$: Angular velocity of the second arm, $\dot{\theta}_3$: Angular velocity of the third arm, T_1 (Torque 1): The control torque applied at the first (bottom) hinge. T_2 (Torque 2): The control torque applied at the third (top) hinge.

Limitation: the linear controller is guaranteed only near the equilibrium; large perturbations may behave differently on the nonlinear plant.

2.3 Variables

Symbol	Meaning	Dimension
x	state vector	6×1
u	input vector (actuator torques)	2×1
A	system (dynamics) matrix	6×6
B	input matrix	6×2
Q	state penalty matrix	6×6
R	control penalty matrix	2×2

Table 1: Variables of the state space model.

3 The Linear-Quadratic Optimal Control Problem

3.1 Background

The linear-quadratic regulator (LQR) finds the control that drives the state to zero while balancing two competing goals: keeping the state small and keeping the control effort small. The designer expresses this balance through two penalty matrices.

3.2 Performance Criterion

For $x_{k+1} = A_d x_k + B_d u_k$ with initial state x_0, the performance criterion (cost) is

$$J = \frac{1}{2} \sum_{k=0}^{\infty} \left(x_k^\top Q \, x_k + u_k^\top R \, u_k \right), \tag{3.1}$$

where

$$Q = Q^\top \succeq 0, \qquad R = R^\top \succ 0. \tag{3.2}$$

The factor $\frac{1}{2}$ is a convention; it scales the cost but does not change the optimal gain or the matrix P. The term $x_k^\top Q x_k$ penalizes deviation from upright (a larger Q gives faster regulation), and $u_k^\top R u_k$ penalizes torque (a larger R gives gentler torques but slower response). The ratio Q/R is the main design knob. The values used are $Q = 10\, I_6$ and $R = 5\, I_2$, so all six states are penalized equally and both inputs are penalized equally.

3.3 Control Law

The optimal control is linear state feedback (Figure 1),

$$u_k = -F \, x_k, \tag{3.3}$$

which gives the closed-loop system

$$x_{k+1} = (A_d - B_d F)\, x_k. \tag{3.4}$$

The optimal cost from x_0 is

$$J^* = \frac{1}{2}\, x_0^\top P^* x_0, \tag{3.5}$$

where P^* is defined in Section 3.5.

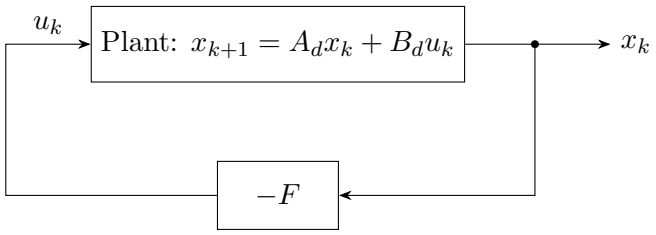


Figure 1: State-feedback control loop. The closed-loop matrix is $A_d - B_d F$.

3.4 Existence Conditions

A unique optimal stabilizing solution requires (A_d, B_d) to be stabilizable and $(A_d, Q^{1/2})$ to be detectable. These follow from the stronger rank conditions

$$\operatorname{rank}\begin{bmatrix} B_d & A_d B_d & \cdots & A_d^{n-1} B_d \end{bmatrix} = n \quad \text{(controllability)}, \tag{3.6}$$

$$\operatorname{rank}\begin{bmatrix} C \\ CA_d \\ \vdots \\ CA_d^{n-1} \end{bmatrix} = n, \quad C = Q^{1/2} \quad \text{(observability)}, \tag{3.7}$$

checked in MATLAB with `rank(ctrb(Ad,Bd))` and `rank(obsv(Ad,sqrtm(Q)))`, with $n = 6$. Controllability means any state can be reached from any other with a suitable input; observability with $C = Q^{1/2}$ means every state that is penalized in the cost is visible to it. Because numerical rank is fragile, the singular values of the controllability matrix are also reported.

3.5 Dynamic Programming and the DARE

Background. Bellman's principle of optimality says the optimal cost from a state equals the best one-step cost plus the optimal cost from the next state. For the LQ problem the optimal cost-to-go is quadratic,

$$V(x) = \frac{1}{2}\, x^\top P x, \qquad P = P^\top \succeq 0, \tag{3.8}$$

and satisfies the Bellman (difference) equation

$$V(x_k) = \min_u \left[\tfrac{1}{2} x_k^\top Q x_k + \tfrac{1}{2} u^\top R u + V(x_{k+1}) \right], \quad x_{k+1} = A_d x_k + B_d u. \tag{3.9}$$

(In continuous time the corresponding equation is the Hamilton–Jacobi–Bellman differential equation.)

Derivation. Substituting (3.8) into (3.9) and cancelling the factor $\frac{1}{2}$,

$$x_k^\top P x_k = \min_u \Big[x_k^\top Q x_k + u^\top R u + (A_d x_k + B_d u)^\top P (A_d x_k + B_d u) \Big].$$

Setting the gradient with respect to u to zero gives

$$R u + B_d^\top P (A_d x_k + B_d u) = 0 \implies (R + B_d^\top P B_d)\, u = -B_d^\top P A_d\, x_k, \tag{3.10}$$

so the optimal control is $u = -F x_k$ with

$$F = (R + B_d^\top P B_d)^{-1} B_d^\top P A_d. \tag{3.11}$$

Substituting this F back into the Bellman equation and requiring it to hold for all x_k gives the **discrete algebraic Riccati equation (DARE)**

$$P = A_d^\top P A_d - A_d^\top P B_d (R + B_d^\top P B_d)^{-1} B_d^\top P A_d + Q. \tag{3.12}$$

An equivalent form, useful in Section 5, is

$$P = (A_d - B_d F)^\top P (A_d - B_d F) + Q + F^\top R F. \tag{3.13}$$

The *stabilizing solution* P^* is the one for which $A_d - B_d F^*$ is asymptotically stable (all eigenvalues inside the unit circle); under the conditions of the previous subsection it exists and is unique. The DARE is nonlinear in P, which motivates the iterative approach below. MATLAB solves it directly with `[Fopt,Popt] = dlqr(Ad,Bd,Q,R)`.

4 The Lyapunov Equation and Policy Evaluation

4.1 Background: Lyapunov Stability

Lyapunov's idea is to certify stability with an energy-like function that decreases along trajectories. For $x_{k+1} = A_{\mathrm{cl}} x_k$ take $V(x) = x^\top P x$ with $P \succ 0$. Then

$$V(x_{k+1}) - V(x_k) = x_k^\top \big(A_{\mathrm{cl}}^\top P A_{\mathrm{cl}} - P \big) x_k < 0 \tag{4.1}$$

for $x_k \neq 0$ whenever

$$A_{\mathrm{cl}}^\top P\, A_{\mathrm{cl}} - P = -W, \qquad W \succ 0, \tag{4.2}$$

which is the **discrete Lyapunov equation**.

Theorem 1. A_{cl} is asymptotically stable if and only if (4.2) has a unique solution $P \succ 0$ for every $W \succ 0$.

In MATLAB, `P = dlyap(Acl', W)`.

4.2 Cost of a Fixed Gain

Let F be any stabilizing gain, $A_F = A_d - B_d F$, and $u_k = -F x_k$. Then the cost (3.1) becomes

$$J(F) = \frac{1}{2} \sum_{k=0}^{\infty} x_k^\top \big(Q + F^\top R F \big) x_k = \frac{1}{2}\, x_0^\top P_F\, x_0, \tag{4.3}$$

with

$$P_F = \sum_{j=0}^{\infty} \left(A_F^\top\right)^j \left(Q + F^\top R F\right) A_F^j, \tag{4.4}$$

which converges because A_F is asymptotically stable. Separating the $j = 0$ term gives $P_F = Q + F^\top R F + A_F^\top P_F A_F$, that is,

$$P_F = A_F^\top P_F A_F + Q + F^\top R F. \tag{4.5}$$

This is a Lyapunov equation (4.2) with $W = Q + F^\top R F$. Its purpose in the algorithm is **policy evaluation**: solving it gives the exact cost $\frac{1}{2} x_0^\top P_F x_0$ of using gain F without simulating the system.

5 Hewer's Algorithm

5.1 Background: Policy Iteration

Policy iteration alternates two steps: *evaluate* the current policy, then *improve* it by acting greedily with respect to that evaluation. For the LQ problem Hewer (1971) showed that this iteration converges to the optimal gain [3, 4].

5.2 Derivation

Policy evaluation. Given a stabilizing gain F_k and $A_k = A_d - B_d F_k$, Eq. (4.5) gives P_k:

$$P_k = A_k^\top P_k A_k + Q + F_k^\top R F_k. \tag{5.1}$$

Policy improvement. Keeping P_k as the estimate of future cost, choose the control that minimizes the one-step objective

$$x^\top Q x + u^\top R u + (A_d x + B_d u)^\top P_k (A_d x + B_d u). \tag{5.2}$$

Setting the gradient with respect to u to zero (as in (3.10)) gives $u = -F_{k+1} x$ with

$$F_{k+1} = (R + B_d^\top P_k B_d)^{-1} B_d^\top P_k A_d. \tag{5.3}$$

Algorithm.

1. Choose F_0 such that $A_d - B_d F_0$ is asymptotically stable.
2. For $k = 0, 1, 2, \ldots$ solve (5.1) for P_k.
3. Compute F_{k+1} from (5.3).
4. Stop when $\|P_k - P_{k-1}\| < \varepsilon$.

The iteration is recursive: F_k determines P_k, and P_k determines F_{k+1}. At a fixed point $F_{k+1} = F_k$, Eq. (5.1) reduces to the DARE (3.13).

5.3 Properties

Under the existence conditions (and starting from a stabilizing F_0):

- every F_k is stabilizing;
- the costs decrease monotonically, $P_0 \succeq P_1 \succeq \cdots \succeq P^*$;
- $P_k \to P^*$ and $F_k \to F^*$.

6 Problem Setup and Implementation

6.1 Model and Checks

- Model A, B from [1] and penalties $Q = 10I_6$, $R = 5I_2$ (Appendix B).
- Discretization with $T = 0.1$ s: A_d, B_d in Appendix B.
- Open-loop eigenvalues of A: 20.634, 1.729 (unstable); -41.101, -25.963, $-0.340 \pm 0.802j$ (stable). Eigenvalue magnitudes of A_d: 7.873, 1.189 (both > 1, matching the two unstable continuous eigenvalues via (**??**)), 0.9666 (twice), 0.0746, 0.0164.
- Controllability (3.6): rank = 6. The smallest singular value of the controllability matrix is 9.65×10^{-4} (largest 3.10×10^{4}, condition number about 3.2×10^{7}), so the pair is controllable but poorly conditioned.
- Observability (3.7): rank = 6.

Both ranks equal 6, so the assumptions for the existence of the LQ optimal control hold.

6.2 Initial Stabilizing Gain

The algorithm needs F_0 with $A_d - B_d F_0$ stable. We place the closed-loop poles inside the unit circle,

$$\operatorname{eig}(A_d - B_d F_0) = \{p_1, \ldots, p_6\}, \qquad |p_i| < 1, \tag{6.1}$$

using MATLAB's `place`, which returns the gain that puts the eigenvalues of $A_d - B_d F_0$ at the desired locations. Repeated poles are limited to $\operatorname{rank}(B_d) = 2$, so distinct poles are used: the continuous poles $-1, -2, \ldots, -6$ mapped through $z = e^{sT}$ give 0.9048, 0.8187, 0.7408, 0.6703, 0.6065, 0.5488. The closed-loop matrix $A_d - B_d F_0$ has exactly these eigenvalues, so $\max |\lambda| = 0.9048 < 1$. Pole choice affects F_0 and the early iterations but not the limit P^*.

6.3 Initial Condition and Metrics

The initial state is $x_0 = [1\ 1\ 1\ 1\ 1\ 1]^\top$, a unit perturbation of every state. Performance of the gain F_k is measured by

$$J_k = \frac{1}{2}\, x_0^\top P_k x_0, \qquad J_0 \geq J_1 \geq \cdots \geq J^*. \tag{6.2}$$

In the implementation $k = 0$ is the pole-placement gain F_0, and $k = 1, \ldots, 6$ are the gains produced by the Hewer updates. Six iterations are run, which is enough for $\|P_k - P_{k-1}\|$ to fall below any practical tolerance.

7 Results

7.1 Convergence Table

Table 2 shows how the approximate performance values J_k converge to the optimal value J^* computed by `dlqr`.

k	J_k
0	6.5689×10^6
1	3.6434×10^5
2	51035
3	33663
4	31157
5	31091
6	31091
`dlqr` (J^*)	31090.76

Table 2: Convergence of the Hewer iterations to the optimal performance value.

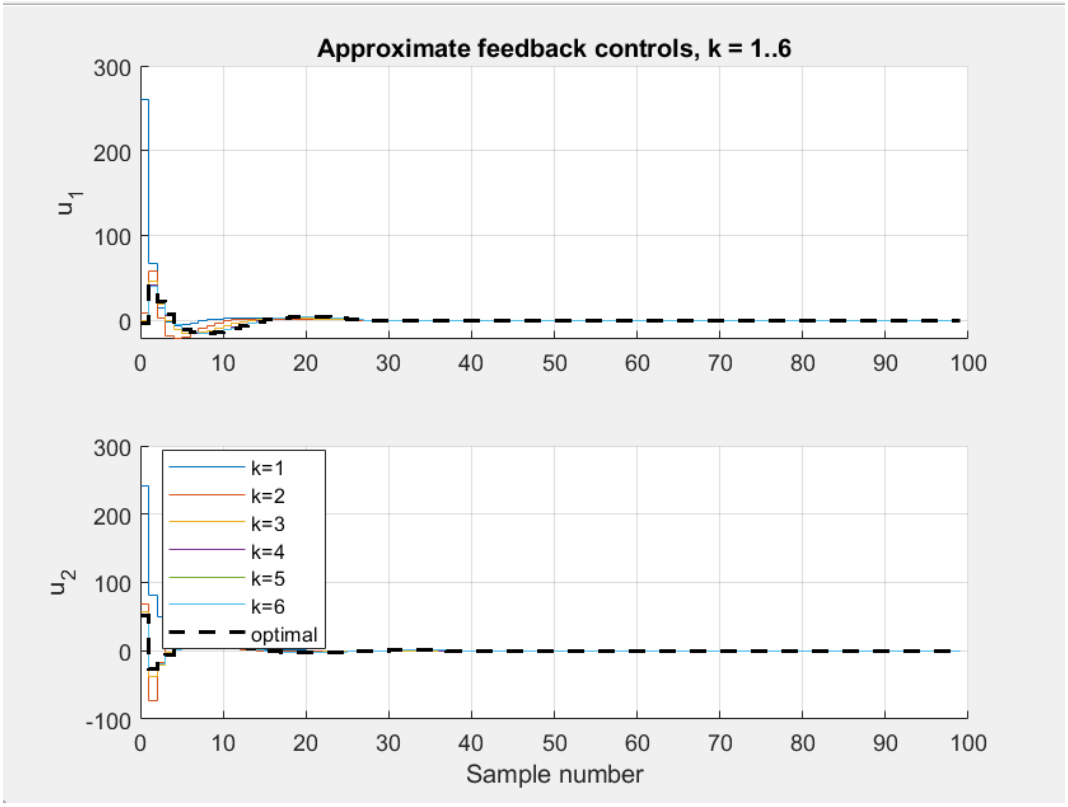


Figure 2: Approximate feedback controls u_1 and u_2 for $k = 1, \ldots, 6$ compared with the optimal control from `dlqr` (dashed).

7.2 Controls and Trajectories

Figure 2 shows both control inputs for $k = 1, \ldots, 6$ together with the optimal control.

The states with the largest peak values at $k = 1$ are x_4, x_5, and x_6, so these are shown as the worst-performing states (Figures 3–5).

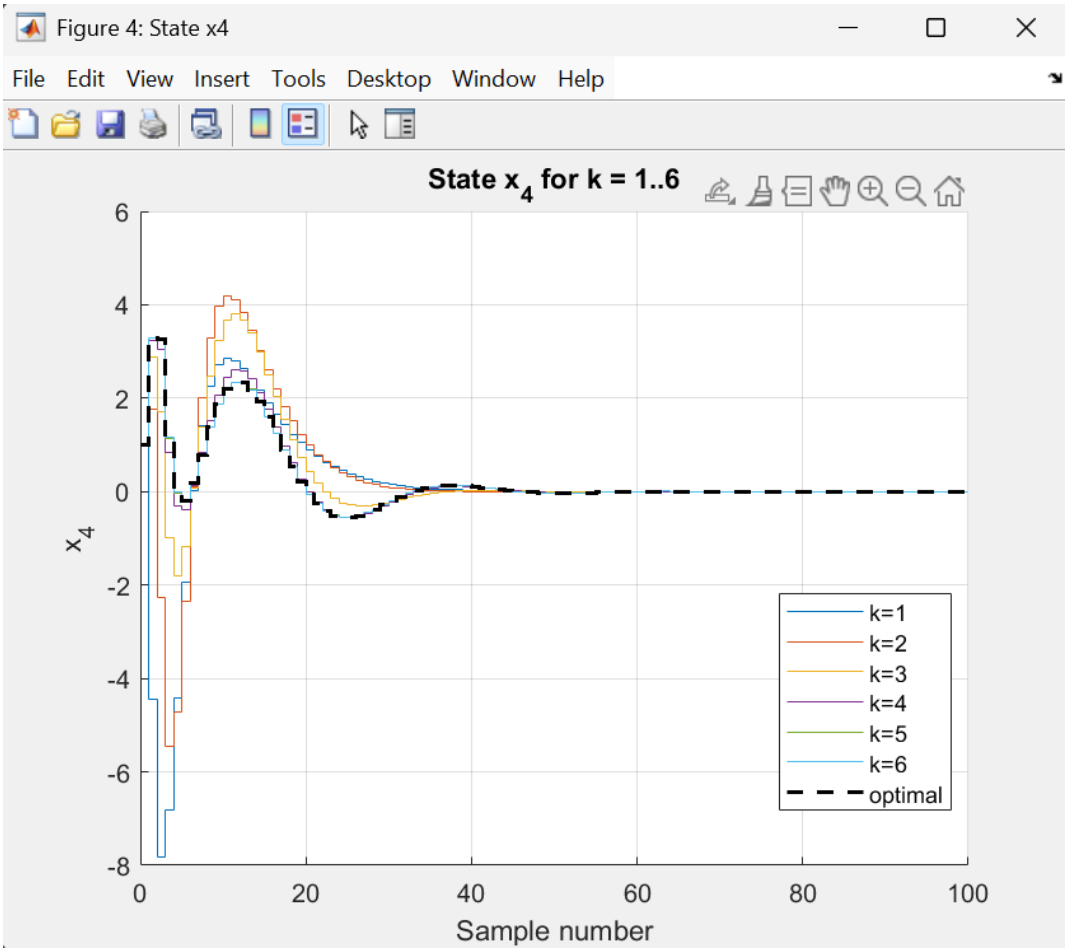


Figure 3: State x_4 for $k = 1, \ldots, 6$ and the optimal gain (dashed).

8 Discussion

- **Convergence.** The cost drops from 6.57×10^6 at $k = 0$ to about 3.6×10^5 after one iteration, to 5.1×10^4, 3.4×10^4, and 3.1157×10^4 at $k = 2, 3, 4$, and is within display precision of $J^* = 31090.76$ by $k = 5$. The first iterations make the largest gains because F_0 is chosen only for stability and ignores the cost. Close to the solution, the error shrinks very fast, consistent with the quadratic rate in (**??**).

- **Stability.** Every F_k yields a stable closed loop, so stability is preserved at each iteration.

- **Worst states.** x_4, x_5, and x_6 show the largest peaks for the early gains ($k = 1, 2$) and the slowest decay, then converge to the optimal trajectory by $k \approx 4$–6. The large initial transients reflect the aggressive, cost-blind gain F_0 and the coupling between links.

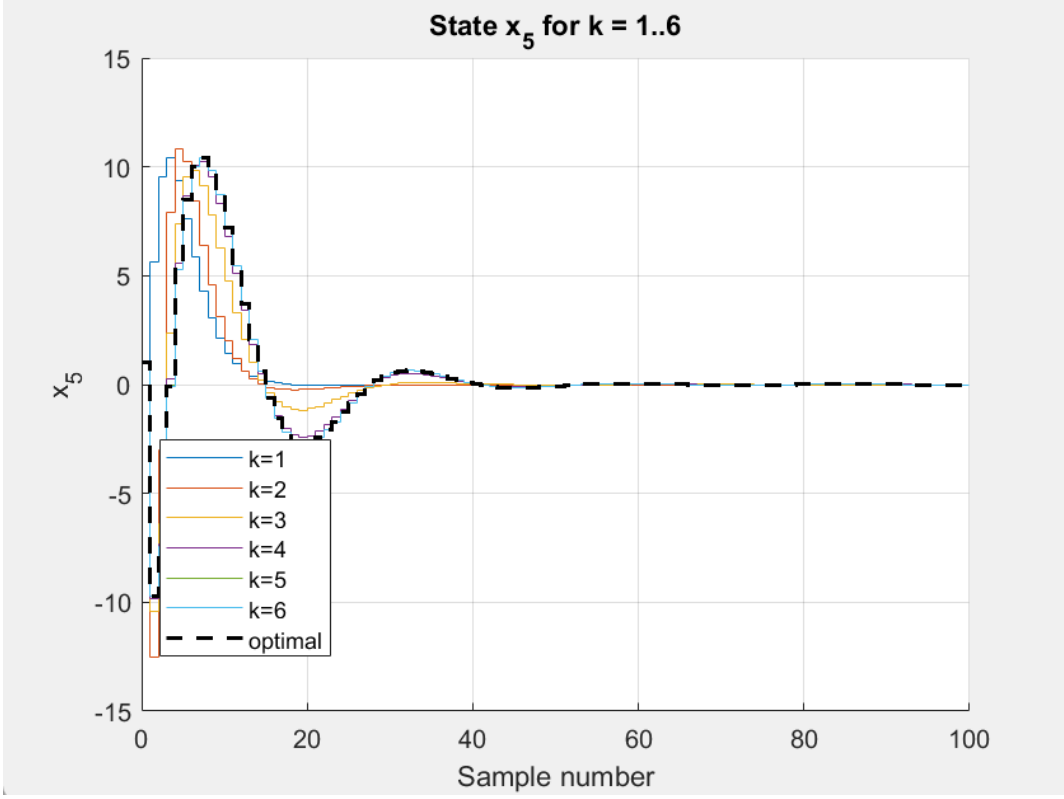


Figure 4: State x_5 for $k = 1, \ldots, 6$ and the optimal gain (dashed).

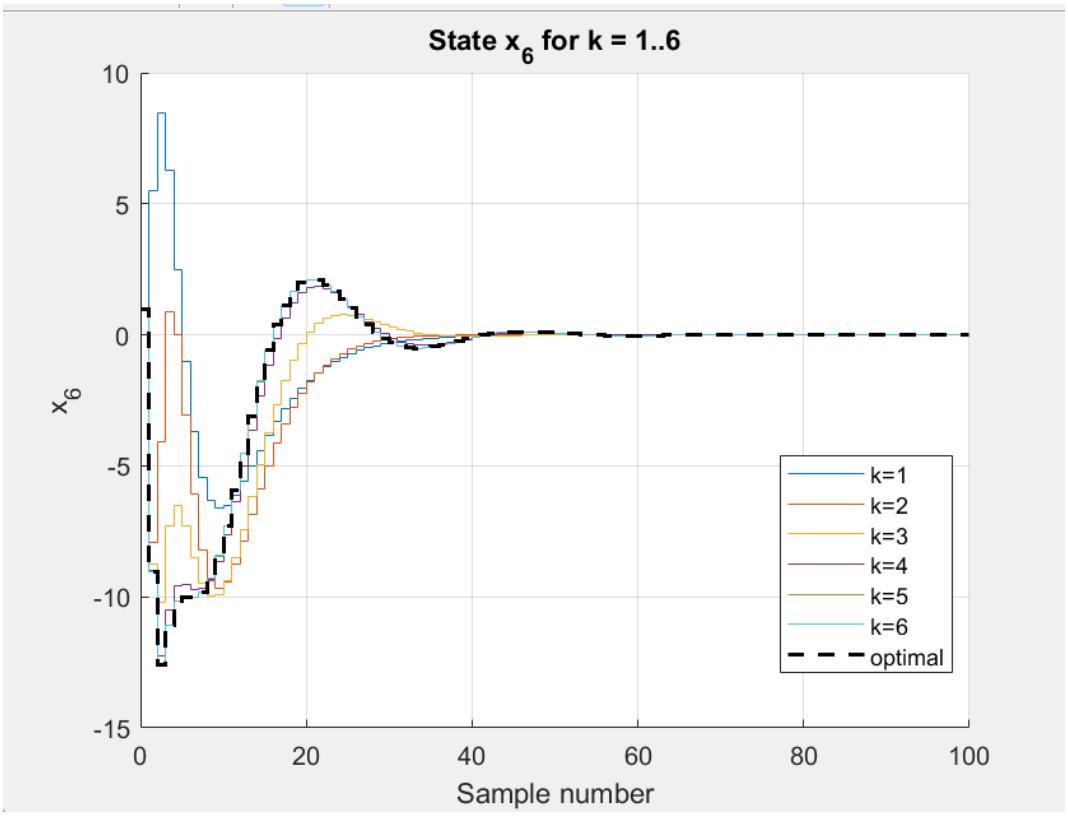


Figure 5: State x_6 for $k = 1, \ldots, 6$ and the optimal gain (dashed).

- **Effect of F_0.** A different pole choice changes the early iterates (and the value of J_0) but not the limit P^*.
- **Q/R tradeoff.** A larger Q shortens settling at the price of larger torques; R acts oppositely.
- **Conditioning.** The controllability matrix has a condition number of about 3×10^7, so the problem is numerically delicate, yet the algorithm converged without difficulty.
- **Limitations.** The model is linearized, so guarantees are local; the design assumes full state feedback and a known model.

8.1 Other Controllers

- **Pole placement:** simple, but gives no guidance for optimality.
- **LQG (LQR with a Kalman filter):** when not all states are measured.
- **LQI:** adds integral action for tracking and disturbance rejection.
- **Model predictive control:** handles torque limits, at higher computational cost.
- **H_∞ / robust control:** addresses model uncertainty.
- **Nonlinear control** (feedback linearization, sliding mode): for large angles.
- **Model-free RL** (Q-learning, adaptive dynamic programming): needs no A or B, unlike Hewer's algorithm.

9 Conclusions

Hewer's algorithm, started from a pole-placement gain F_0, converged to the DARE solution (3.12) computed by `dlqr` in six iterations. The cost fell monotonically from 6.57×10^6 to the optimal value $J^* \approx 31090.76$, every intermediate gain was stabilizing, and the intermediate controls and state trajectories approached the optimal ones. The fast final convergence is consistent with the quadratic rate of Newton's method, which the Hewer iteration implements.

References

[1] https://acta.fih.upt.ro/pdf/2021-2/ACTA-2021-2-08.pdf

[2] https://iopscience.iop.org/article/10.1088/1742-6596/1007/1/012068/pdf

[3] Reinforcement Learning for Engineering MATLAB Design Projects - Modules: By Zoran Gajic and Lingyi Xu

[4] MATLAB design projects book, Chapter 2 and Section 3.5, Demos 3.1 and 3.2.

[5] Linear System Theory By Wilson J. Rugh

A MATLAB Code

```
%% Hewer algorithm (discrete-time LQ RL) for the triple inverted pendulum
clear; clc; close all;

%% 1. Model from [1] and discretization
A = [-57.75 -14.42 -15.82 -16.26  -5.5   -7;
```

```
      -25.3   -43     -12.7   -11     -13.6    -5;
      178      88.6    55.6    57.3    30.2    25;
      -22.9   -10      -0.14   -0.08   -0.04   -0.008;
      -26     -37.5    -3.6    -0.15   -0.13   -0.033;
       -0.83   -8.2    -9.25   -0.01   -0.04   -0.02];
B = [ 1.559  -0.8;
      -4       3.8;
       0.65   -2.4;
       0 0; 0 0; 0 0];
C = eye(6);  D = zeros(6,2);

Ts   = 0.1;
sysd = c2d(ss(A,B,C,D), Ts);
[Ad, Bd] = ssdata(sysd);
n = size(Ad,1);
m = size(Bd,2);

Q  = 10*eye(n);           % state penalty
R  = 5*eye(m);            % control penalty
x0 = ones(n,1);           % initial condition

%% 2. Existence checks (controllability / observability)
CO = ctrb(Ad,Bd);
OM = obsv(Ad,sqrtm(Q));  % (Ad, Q^(1/2)) observable
fprintf('rank(ctrb) = %d, rank(obsv) = %d, n = %d\n', rank(CO), rank(OM), n);
fprintf('smallest singular value of ctrb: %.3e\n', min(svd(CO)));

%% 3. Optimal solution (reference)
[Fopt, Popt] = dlqr(Ad, Bd, Q, R);
Jopt = 0.5*x0'*Popt*x0;

%% 4. Initial stabilizing gain F0 via pole placement
poles_c = [-1 -2 -3 -4 -5 -6];
F = place(Ad, Bd, exp(poles_c*Ts));     % discrete poles inside unit circle
fprintf('max |eig(Ad-Bd*F0)| = %.4f\n', max(abs(eig(Ad-Bd*F))));

%% 5. Hewer iterations: k = 0 is F0, k = 1..Kmax are the improved gains


Kmax = 6;  Nt = 100;
Fk = zeros(m,n,Kmax+1);
Pk = zeros(n,n,Kmax+1);
J  = zeros(Kmax+1,1);
maxeig = J;
X  = zeros(n,Nt+1,Kmax+1);
U = zeros(m,Nt,Kmax+1);

for k = 0:Kmax
    Acl = Ad - Bd*F;
    P   = dlyap(Acl', Q + F'*R*F);              % policy evaluation
    idx = k+1;
    Fk(:,:,idx) = F;
    Pk(:,:,idx) = P;
    J(idx)      = 0.5*x0'*P*x0;

    maxeig(idx) = max(abs(eig(Acl)));

    X(:,1,idx) = x0;                            % closed-loop simulation
    for j = 1:Nt
```

```
        U(:,j,idx)   = -F*X(:,j,idx);
        X(:,j+1,idx) = Acl*X(:,j,idx);
    end

    F = (R + Bd'*P*Bd) \ (Bd'*P*Ad);             % policy improvement
end

% optimal trajectory for comparison
Xo = zeros(n,Nt+1);
Uo = zeros(m,Nt);
Xo(:,1) = x0;
for j = 1:Nt
    Uo(:,j)   = -Fopt*Xo(:,j);
    Xo(:,j+1) = (Ad - Bd*Fopt)*Xo(:,j);
end

%% 6. Convergence table
k = (0:Kmax)';
convTable = table(k, J, 'VariableNames', {'k','J_k'});
disp(convTable);
fprintf('Jopt = %.6f\n', Jopt);

%% 7. Figures
kk = 0:Nt-1;  cols = lines(Kmax);

% 7a. Both control inputs for k = 1..6 (+ optimal)
figure('Name','Controls');
for i = 1:m
    subplot(m,1,i);
    hold on;
    grid on;
    for q = 1:Kmax
        stairs(kk, U(i,:,q+1), 'Color', cols(q,:));
    end
    stairs(kk, Uo(i,:), 'k--', 'LineWidth', 1.5);
    ylabel(sprintf('u_%d',i));
    if i == 1, title('Approximate feedback controls , k = 1..6'); end
    if i == m, xlabel('Sample number'); end
end
legend([compose('k=%d',1:Kmax), {'optimal'}], 'Location','best');

% 7b. states improvment (largest peak at k = 1), k = 1..6 (+ optimal)
[~,order] = sort(max(abs(X(:,:,2)),[],2), 'descend');
worst = order(1:3);
for s = worst'
    figure('Name',sprintf('State x%d',s));
    hold on;
    grid on;
    for q = 1:Kmax
        stairs(0:Nt, X(s,:,q+1), 'Color', cols(q,:));
    end
    stairs(0:Nt, Xo(s,:), 'k--', 'LineWidth', 1.5);
    xlabel('Sample number');
    ylabel(sprintf('x_%d',s));
    title(sprintf('State x_%d for k = 1..6',s));
    legend([compose('k=%d',1:Kmax), {'optimal'}], 'Location','best');
end

% 7c. Convergence of performance and of P
```

```
figure('Name','Convergence');
plot(k, J, '*-');
hold on;
yline(Jopt,'k--','J_{opt}');
grid on;
xlabel('Iteration k');
ylabel('J_k');
title('Hewer algorithm convergence');
```

Symbol	Meaning
A, B	continuous-time system and input matrices
A_d, B_d	discrete-time system and input matrices
T	sampling period (0.1 s)
x_k, u_k	state and input at sample k
Q, R	state and control penalty matrices
J	performance criterion (cost)
P	Riccati / Lyapunov matrix; P^* stabilizing DARE solution
F	state-feedback gain; F^* optimal gain
F_k, P_k	Hewer iterates (gain and cost matrix at iteration k)
$A_k = A_d - B_d F_k$	closed-loop matrix at iteration k
W	right-hand side of the Lyapunov equation
$\succeq$, $\succ$	positive semidefinite, positive definite

Table 3: Notation used in this report.

B Model Data

Matrices are shown to four significant digits. $Q = 10\, I_6$ and $R = 5\, I_2$.

$$A = \begin{bmatrix} -57.75 & -14.42 & -15.82 & -16.26 & -5.5 & -7 \\ -25.3 & -43 & -12.7 & -11 & -13.6 & -5 \\ 178 & 88.6 & 55.6 & 57.3 & 30.2 & 25 \\ -22.9 & -10 & -0.14 & -0.08 & -0.04 & -0.008 \\ -26 & -37.5 & -3.6 & -0.15 & -0.13 & -0.033 \\ -0.83 & -8.2 & -9.25 & -0.01 & -0.04 & -0.02 \end{bmatrix}, \quad B = \begin{bmatrix} 1.559 & -0.8 \\ -4 & 3.8 \\ 0.65 & -2.4 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$A_d = \begin{bmatrix} -2.285 & -0.5655 & -1.467 & -2.083 & -1.06 & -0.9122 \\ -2.106 & -0.2143 & -1.403 & -1.929 & -1.405 & -0.8563 \\ 10.83 & 2.844 & 6.438 & 7.959 & 4.416 & 3.499 \\ 2.576 & 0.6214 & 2.002 & 3.587 & 1.43 & 1.137 \\ 3.348 & -0.3333 & 2.592 & 3.671 & 3.55 & 1.627 \\ -4.337 & -1.835 & -2.57 & -2.277 & -1.053 & 0.003976 \end{bmatrix}, \quad B_d = \begin{bmatrix} -0.02322 & 0.08568 \\ -0.2367 & 0.2644 \\ 0.3283 & -0.5283 \\ 0.07678 & -0.1487 \\ 0.3692 & -0.4356 \\ 0.01345 & 0.05851 \end{bmatrix}$$

$$P^* = \begin{bmatrix} 9933 & 4196 & 4716 & 2972 & 1214 & 828 \\ 4196 & 1830 & 1951 & 1090 & 428.5 & 269.6 \\ 4716 & 1951 & 2324 & 1729 & 668.9 & 527.7 \\ 2972 & 1090 & 1729 & 2406 & 709.8 & 925.4 \\ 1214 & 428.5 & 668.9 & 709.8 & 326.2 & 251.9 \\ 828 & 269.6 & 527.7 & 925.4 & 251.9 & 406.8 \end{bmatrix}$$

$$F^* = \begin{bmatrix} -0.9948 & -2.095 & 0.4003 & 2.3 & 2.737 & 1.435 \\ -15.18 & -4.162 & -9.292 & -12.56 & -6.035 & -5.081 \end{bmatrix}$$